PURBANCHAL UNIVERSITY

HIMALAYAN WHITEHOUSE INTERNATIONAL COLLEGE

Putalisadak, Kathmandu

A

Project-VIII Proposal Report

on

Kinyo: A Multi-Tenant E-Commerce Platform

8th Semester Apprenticeship Project

Submitted by

Hikmat Baniya [15]

Nishan Neupane [23]

Submitted to the Department of Science and Technology in Partial Fulfilment of the Requirements for the Degree of Bachelor of Information Technology

Department of Science and Technology

Kathmandu, Nepal

September, 2026

Abstract

Kinyo is a multi-tenant e-commerce platform that allows many independent sellers to create, customise and operate their own branded online storefronts from a single deployed application. Most small retailers in Nepal sell online through social media pages and messaging applications, where the catalogue is a series of image posts, the order book is a chat thread and stock is tracked from memory. Existing hosted platforms solve this but charge subscriptions in foreign currency, while existing open-source platforms are built around one merchant per deployment and require a developer to install and maintain.

The proposed system is built as a three-tier web application. A FastAPI service exposes a versioned REST interface over a single PostgreSQL database in which every tenant-owned table carries a tenant identifier, and a Next.js application resolves the store from the HTTP Host header so that storefronts are reachable both at a platform subdomain and at a custom domain owned by the seller. Isolation between tenants is enforced in one data-access layer and backed by row-level security in the database, and is verified by a dedicated group of cross-tenant test cases rather than assumed. The platform covers store provisioning, domain mapping, storefront themes, the product catalogue with variants and collections, variant-level inventory, cart, checkout, the order lifecycle, discounts, shipping zones, sales reporting and a platform administration console. Orders are settled by cash on delivery; online payment gateway integration is outside the scope of this project.

Keywords: Multi-Tenant Architecture, E-Commerce Platform, Software as a Service, FastAPI, Next.js, PostgreSQL, Tenant Isolation, Storefront Routing, Inventory Management, Cash on Delivery.

Table of Contents

List of Figures

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CDN	Content Delivery Network
CNAME	Canonical Name (Domain Name System record)
COD	Cash on Delivery
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DNS	Domain Name System
ER	Entity Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PK	Primary Key
FK	Foreign Key
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
RLS	Row-Level Security
SaaS	Software as a Service
SDLC	Software Development Life Cycle
SEO	Search Engine Optimisation
SKU	Stock Keeping Unit
SME	Small and Medium Enterprise
SQL	Structured Query Language
SSR	Server-Side Rendering
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience

Chapter 1: Introduction

1.1 Background

Retail trade in Nepal is dominated by small and medium enterprises that operate from a single shop and sell to a local customer base. Over the past decade a large share of these sellers has moved part of that trade online, but the move has been made almost entirely through general-purpose social media pages and messaging applications rather than through owned sales channels. A seller typically publishes product photographs on a social media page, negotiates price and availability in private messages, records the resulting orders in a notebook or a spreadsheet, and arranges delivery by telephone. The country commercial guidance published for Nepal notes that online retail is expanding quickly while the supporting commercial infrastructure remains uneven (International Trade Administration, 2024).

This way of working has clear shortcomings. The seller does not control the presentation of the catalogue, cannot show reliable stock levels, and has no record of an order beyond a chat history that is easily lost. Prices, discounts and delivery charges are re-negotiated for every customer. Because the storefront is a page inside someone else's platform, the seller cannot be found through ordinary web search, cannot build a branded identity, and cannot export a customer list.

The established alternative is a hosted commerce platform. Systems such as Shopify and Wix eCommerce allow a seller to create a branded online store without writing code, and open-source systems such as WooCommerce, Saleor and Medusa allow the same outcome for a seller who can pay for development and hosting (Shopify, 2024; Saleor Commerce, 2024). These systems are mature, but each of them assumes either a recurring subscription paid in foreign currency or a level of technical skill that an average Nepali shop owner does not have.

The project proposed here, named Kinyo, is a multi-tenant e-commerce platform on which many independent sellers operate their own branded storefronts from a single deployed application. Multi-tenancy is the architectural practice of serving many customer organisations, called tenants, from one running instance of an application and one database, with the data of each tenant isolated from the others (Chong & Carraro, 2006). Applying that practice here allows the cost of hosting, maintenance and upgrades to be shared across all sellers, which is what makes an affordable, locally operated storefront service possible.

1.2 Problem Statement

Independent retailers in Nepal who wish to sell online have no affordable way to obtain and operate a storefront that they control. The current process depends on social media pages and manual record keeping: the catalogue is a series of image posts, the order book is a chat thread, and stock is tracked from memory. As a consequence, orders are lost or duplicated, items are sold after they are out of stock, pricing and delivery charges are inconsistent between customers, and no sales history exists from which the seller could plan purchasing.

The parties that bear the cost of this gap are the sellers, who lose sales and spend unpaid hours on manual coordination, and their customers, who cannot see accurate availability, cannot check the status of an order, and have no record of what they agreed to buy. Existing marketplace applications do not close the gap, because a seller listed inside a marketplace does not own the storefront, the customer relationship or the presentation of the brand. Existing hosted platforms do not close it either, because their subscription pricing and configuration effort place them beyond the reach of a single-shop retailer.

There is currently no system that provides a Nepali independent seller with a self-service branded storefront, an accurate catalogue with variant-level stock control, and a recorded order

lifecycle, at a cost that a single-shop business can sustain. The absence of such a system keeps small retailers dependent on manual, error-prone processes and prevents them from competing online with larger sellers.

1.3 Objectives

1.3.1 General Objective

To design, develop and deploy Kinyo, a multi-tenant e-commerce platform that enables independent sellers to create, customise and operate their own branded online storefronts from a single shared application.

1.3.2 Specific Objectives

The specific objectives of the project are the following.

1. To analyse the requirements of independent sellers and their customers for online catalogue management, storefront presentation and order handling.
2. To design a multi-tenant data model and a three-tier architecture that isolates every tenant-owned record and resolves each storefront from its host name.
3. To deploy the platform as a web application that serves seller dashboards and customer storefronts over subdomain and custom-domain addresses.

1.4 Project Scope

The boundaries of the project are defined below. The system is a storefront and order-management platform; it is not an accounting system, a marketplace or a logistics system.

- **Features included:** seller registration and role-based staff access; store provisioning; subdomain and custom-domain mapping; storefront theme selection and customisation; product catalogue with variants, collections and media; variant-level inventory tracking; shopping cart for registered and guest customers; checkout and order placement; order

status and fulfilment tracking; discount codes; shipping zones and rates; sales reporting for sellers; and a platform administration console for approving and suspending stores.

- **Target users:** independent retailers and their staff who sell physical goods; the customers who buy from those storefronts; and the platform administrators who operate Kinyo.
- **Scope of functionality:** the system will manage storefronts, catalogues, inventory, carts and the order lifecycle. It will not perform accounting, tax filing or bookkeeping, will not integrate an online payment gateway, and will not manage warehousing or courier dispatch. Orders are completed using cash on delivery, and the payment status is recorded manually by the seller. Online payment integration is identified as future work and is deliberately excluded so that the multi-tenant storefront functionality can be completed and tested within the academic timeframe.
- **Deployment scope:** the system will be delivered as a web application, comprising a browser-based seller dashboard, a browser-based platform administration console, and server-rendered public storefronts. Native mobile applications are out of scope; the storefronts will be responsive and usable on mobile browsers.

Chapter 2: Literature Overview

2.1 Background and Theoretical Framework

The project rests on three bodies of technical practice: multi-tenant application architecture, relational data modelling, and the design of stateless web application programming interfaces. Each is described below together with the reason it was adopted.

Multi-tenancy. A multi-tenant application serves several independent customer organisations from one running instance. Chong and Carraro (2006) distinguish three approaches to isolating tenant data: a separate database for each tenant, a shared database with a separate schema for each tenant, and a shared database with a shared schema in which every tenant-owned row carries a tenant identifier. The three approaches trade isolation against cost: separate databases give the strongest isolation and the highest per-tenant operating cost, while the shared schema gives the lowest cost and requires the application to enforce isolation on every query. Krebs et al. (2012) reach the same conclusion from a performance standpoint and note that the shared-schema form is the only one that scales economically to large numbers of small tenants. Bezemer and Zaidman (2010) add the maintenance argument: a single shared instance means that a defect is fixed once for every tenant, but it also means that a defect in the isolation logic is exposed to every tenant at once. Kinyo adopts the shared-database, shared-schema model, because its target tenants are numerous and individually small, and it treats tenant isolation as a first-class correctness requirement that is tested explicitly rather than assumed.

Relational data modelling. The catalogue, cart and order data of a commerce system are highly relational: a product has many variants, a variant has one stock record, an order has many line items, and each of those line items refers to a variant. The relational model gives a formal basis for representing those associations without duplication and for querying them consistently

(Codd, 1970). The schema described in Section 3.8 is normalised to third normal form, with the deliberate exception that the unit price of a cart or order line is stored on the line rather than read from the variant, so that an order preserves the price that the customer actually agreed to when a product price later changes.

Stateless web APIs. The application layer exposes its behaviour as a REST interface. REST is an architectural style in which each request carries all the information needed to interpret it, resources are addressed by uniform identifiers, and the server keeps no client session state between requests (Fielding, 2000). A stateless interface suits this project because the seller dashboard, the platform console and the server-rendered storefronts are three different clients of the same interface, and because authentication can then be carried in a signed token rather than in server-side session storage.

Access control. Two distinct populations authenticate against the system: platform users, who are sellers, their staff and platform administrators, and storefront customers, who belong to a single tenant. Platform users are authorised through role-based access control, in which permissions are attached to roles and roles are granted to users within a particular tenant, so that the same person may be an owner of one store and a staff member of another. The security requirements for both populations follow the categories of the OWASP Top 10, in particular broken access control and identification failures (Open Web Application Security Project, 2021).

Three-tier organisation. The system is organised into a presentation tier, an application tier and a data tier, described in Section 3.3. This separation was chosen over a single monolithic web application because the presentation tier must render public storefronts for search engine visibility, while the application tier must serve several different clients; keeping them apart allows each to be developed, tested and deployed independently.

2.2 Study of Related Systems and Related Work

Six existing systems relevant to the problem area were reviewed. Each is summarised below by platform, approach, strengths and limitations, and each is cited in the reference list.

1) Shopify

- Platform: hosted software as a service, reached through a web browser.
- Approach: a proprietary multi-tenant platform with a template language that sellers use to theme their storefronts.
- Strengths: self-service store creation, custom domain mapping, a mature checkout and a large application ecosystem.
- Limitations: the subscription is charged in foreign currency, the storefront logic cannot be modified, and there is no integration with local payment or delivery providers.

2) WooCommerce

- Platform: self-hosted, installed as a plugin into a WordPress site.
- Approach: extends a PHP and MySQL content management system with commerce features.
- Strengths: open source, no licence cost, a very large plugin catalogue and complete control of the storefront.
- Limitations: single tenant by design. Each seller needs a separate installation, database and hosting account, which is precisely the barrier this project removes.

3) Adobe Commerce (Magento)

- Platform: self-hosted or vendor-hosted, reached through a web browser.
- Approach: a modular PHP commerce framework with support for several storefronts from one installation.

- Strengths: rich catalogue, pricing and promotion rules, and genuine multi-store capability.
- Limitations: heavy hardware and expertise requirements, and its multi-store support assumes one owning business rather than unrelated tenants who must not see each other's data.

4) Wix eCommerce

- Platform: hosted software as a service.
- Approach: a proprietary multi-tenant website builder with a commerce module bolted on.
- Strengths: drag-and-drop storefront design, with hosting and domain management included.
- Limitations: the storefront structure is fixed by the builder, data export is limited, and the subscription is again priced in foreign currency.

5) Saleor

- Platform: self-hosted, deployed by the merchant or an integrator.
- Approach: a Python and Django core that exposes a GraphQL interface, with the storefront kept as a separate application.
- Strengths: a modern interface-first design, a strong catalogue and sales-channel model, and an open licence.
- Limitations: aimed at development teams. There is no self-service tenant onboarding, and a separate deployment is expected per merchant.

6) Medusa

- Platform: self-hosted, deployed by the merchant or an integrator.

- Approach: a Node.js commerce engine with a headless interface and a separate administration application.
- Strengths: modular services, an open licence, and a flexible order and pricing model.
- Limitations: single-merchant by default. Multi-tenancy has to be added by the integrator, and a developer is required to operate it.

The systems reviewed in Table 1 fall into two groups. The hosted platforms (Shopify, 2024; Wix.com, 2024) already solve the multi-tenant problem: one running system serves many sellers, and a seller can open a store without technical help. Their limitation for this project's users is commercial and contextual rather than technical, namely subscription pricing in foreign currency and the absence of any adaptation to local delivery and settlement practice. The open-source platforms (WooCommerce, 2024; Adobe, 2024; Saleor Commerce, 2024; Medusa, 2024) remove the subscription cost and give full control of the code, but every one of them is built around a single merchant per deployment. Making them serve many independent sellers requires either a separate installation, database and hosting account per seller, or substantial custom development to add tenancy.

The gap that this project addresses lies between those two groups: an open, self-hosted platform that is multi-tenant by design, so that one deployment can onboard many unrelated sellers through self-service, and that is shaped around the way small Nepali retailers actually sell, including cash on delivery as the default settlement method.

2.3 Contribution of the Proposed System

Based on the limitations identified in Section 2.2, the proposed system contributes the following.

- Self-service multi-tenancy on an open stack. Unlike WooCommerce, Saleor and Medusa, which assume one deployment per merchant, Kinyo provisions a new tenant, its subdomain and its storefront from a registration form, with no additional installation or hosting account. A single deployment therefore serves an arbitrary number of sellers.
- Tenant isolation enforced and tested, not assumed. Every tenant-owned table carries a tenant identifier, queries are scoped through a single tenancy layer in the ORM, and cross-tenant access is covered explicitly by the test cases defined in Section 4.3. This responds directly to the maintenance risk that Bezemer and Zaidman (2010) identify in shared-instance systems.
- Host-based storefront routing with custom domains. The presentation tier resolves the tenant from the HTTP Host header, so a storefront is reachable both at a platform subdomain and at a domain owned by the seller. This gives an independent seller the branded presence that a marketplace listing cannot provide.
- A workflow shaped for local practice. The order lifecycle treats cash on delivery as a first-class settlement method with an explicit fulfilment and collection state, rather than as an exception to card payment, and prices are held in a single tenant-level currency.
- An integration that existing systems keep separate. Catalogue, variant-level inventory and the order lifecycle are managed in one application with one data model, so that stock is reserved at the moment an order is created rather than reconciled afterwards from a separate record.

2.4 Functional and Non-Functional Requirements

This section states what the system must do and how well it must perform. The functional requirements describe the operations the system carries out; the non-functional requirements describe the qualities it must exhibit while doing so.

Functional Requirements

1) Authentication and Access Control

- Sellers, staff and platform administrators shall be able to register, log in and log out, with sessions carried by signed tokens.
- A store owner shall be able to invite staff and assign them roles, and each role shall permit only the operations defined for it.
- Storefront customers shall authenticate separately from platform users, because a customer account belongs to one store rather than to the platform.

2) Store Provisioning and Domain Mapping

- A registered seller shall be able to create a store, to which the system assigns a unique slug and a platform subdomain.
- A store owner shall be able to attach a custom domain, which the system verifies before serving the storefront from it.
- A platform administrator shall be able to approve, suspend and reinstate stores.

3) Catalogue and Inventory Management

- Sellers shall be able to create, update, publish and archive products, variants, collections and product media.

- Each variant shall carry its own stock keeping unit and price, and the system shall record stock on hand and reserved stock for each variant.
- The system shall prevent an order that requests more units than are available.

4) Storefront and Shopping Cart

- The system shall resolve the tenant from the request host and render only that tenant's catalogue and theme.
- Registered and guest customers shall be able to add, update and remove cart lines, and the cart shall persist between visits.
- The storefront shall support product search and browsing by collection.

5) Checkout and Order Processing

- Customers shall be able to place an order with a delivery address and a cash on delivery payment method, and shall receive an order number.
- The system shall recompute the subtotal, discount, shipping charge and total on the server at checkout rather than trusting values sent by the client.
- Store staff shall be able to advance an order through its status sequence and record collection of payment on delivery.

6) Discounts, Shipping and Reporting

- Sellers shall be able to define discount codes with validity periods and minimum order values, and shipping zones with associated rates.
- The system shall generate sales and inventory reports for a store on demand, and a platform activity summary for administrators.

7) Error Handling and Validation

- Invalid input shall be rejected with a clear, non-technical message identifying the field at fault.
- A request for a host that does not map to an active store shall return a store-not-found page rather than an application error.
- If stock changes between adding an item to the cart and confirming the order, the customer shall be returned to the cart with the affected lines identified.

Non-Functional Requirements

1) Tenant Isolation

- No request authenticated for one tenant shall be able to read or modify data belonging to another tenant.
- Isolation shall be enforced in a single data-access layer and reinforced by row-level security policies in the database.

2) Security

- Credentials shall be stored only as salted hashes using Argon2, and all traffic shall be served over HTTPS.
- The application shall address the OWASP Top 10 categories, in particular broken access control and identification failures.
- No card or bank data is collected or stored, because no online payment gateway is integrated.

3) Performance

- A storefront catalogue page shall be returned within 3 seconds under the documented test conditions.
- Storefront responses shall be cached so that repeated catalogue reads do not reach the database on every request.

4) Usability

- A seller shall be able to create a store and publish a first product without training or developer assistance.
- The storefront shall be responsive and usable on a mobile browser.

5) Scalability

- Adding a further seller shall require no additional deployment, database or hosting account.
- The application tier shall be stateless so that further instances can be added behind a load balancer.

6) Maintainability

- Code shall be modular, documented and version-controlled in a Git repository.
- Database changes shall be applied through versioned migrations committed alongside the code that requires them.

2.5 Feasibility Study

The feasibility study evaluates whether the proposed platform can be developed and deployed within the available technical, operational, economic, schedule and legal constraints.

1) Technical Feasibility

- Every required technology is mature, openly licensed and comprehensively documented: FastAPI for the application tier, Next.js for the presentation tier, PostgreSQL for storage, and SQLAlchemy with Alembic for data access and migrations.
- The team already has working knowledge of Python, JavaScript and relational databases.
- A standard laptop with 8 GB of memory is sufficient for development, because the workload is dominated by database access rather than computation. No specialised hardware is required.
- The shared-database multi-tenant pattern adopted is well described in the literature, so the highest-risk part of the design is not being invented from scratch.

2) Operational Feasibility

- The intended users already sell online through social media, so the concepts of a catalogue, an order and a delivery address are familiar.
- The seller dashboard replaces a manual notebook rather than an existing software system, which removes the migration effort that normally blocks adoption.
- Store provisioning is self-service, so a seller can open a store from a registration form without contacting the platform operator.
- Cash on delivery is retained as the settlement method because it is what these sellers and their customers already use.

3) Economic Feasibility

- Every tool in the stack is free and open source, and no commercial licence is required at any stage.

- Development and demonstration hosting are covered by free tiers for the application and a small managed PostgreSQL instance.
- The only unavoidable cost is a domain name for the demonstration deployment, which is a small annual fee.
- For the platform operator, the shared-database design means the marginal cost of an additional seller is the cost of the rows that seller creates, rather than the cost of a further deployment.

4) Schedule Feasibility

- The project is scheduled over nineteen weeks, from Ashadh to Kartik, as set out in Figure 2.
- Requirement analysis, design, development, integration, testing, deployment and documentation are treated as separate activities, and testing is allocated its own period rather than being absorbed into development.
- Tenant isolation, the highest-risk element of the design, is built in the first development iteration so that it is exercised by every later increment.

5) Legal and Ethical Feasibility

- The system stores personal data of storefront customers, namely name, contact details and delivery address. That data is collected only for order fulfilment, is scoped to the tenant that collected it, and is transmitted over HTTPS.
- No card or bank data is collected or stored, because no online payment gateway is integrated, which keeps the project outside the scope of payment card compliance.
- All third-party libraries used are released under permissive open-source licences that allow academic and commercial use.

Based on the above analysis, the project is determined to be feasible within the given academic timeframe, available tools and team capability.

Chapter 3: System Design and Methodology

3.1 Software Development Life Cycle

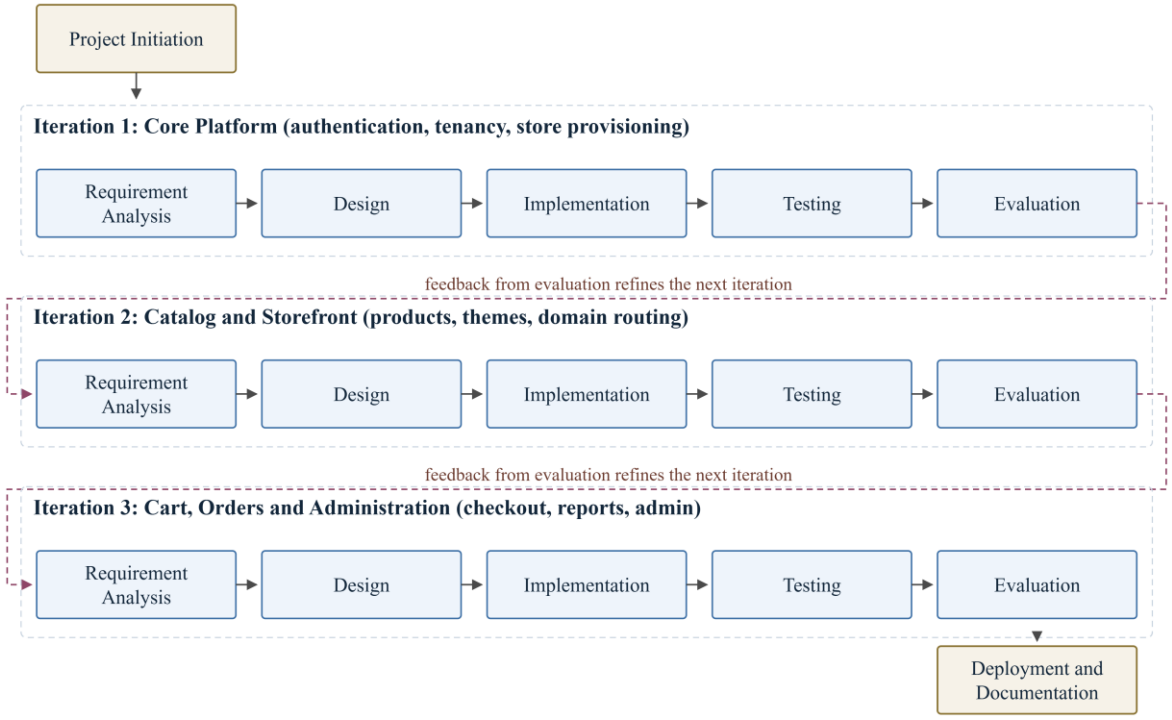
The iterative and incremental model has been selected for this project. In this model the system is built in a series of iterations, each of which passes through requirement analysis, design, implementation, testing and evaluation, and each of which delivers a working increment of the system (Sommerville, 2016).

The model was chosen for three reasons specific to this project. First, the requirements of the seller dashboard are expected to be refined through repeated feedback from prospective sellers, and an iterative model allows that feedback to be absorbed without restarting the design. Second, the system decomposes naturally into increments that can be built and tested independently: the tenancy and authentication core, the catalogue and storefront, and finally the cart, order and administration modules. Third, the multi-tenant isolation logic is the highest-risk element of the design, and building it in the first iteration means it is exercised by every subsequent increment rather than being validated only at the end.

A purely sequential waterfall model was rejected because it would defer all testing of tenant isolation until after the whole system was written, and an agile process with continuous customer involvement was rejected because the project has no permanently available product owner. Figure 1 shows the model as it will be applied.

Figure 1

SDLC Model: Iterative and Incremental Development



3.2 Project Timeline

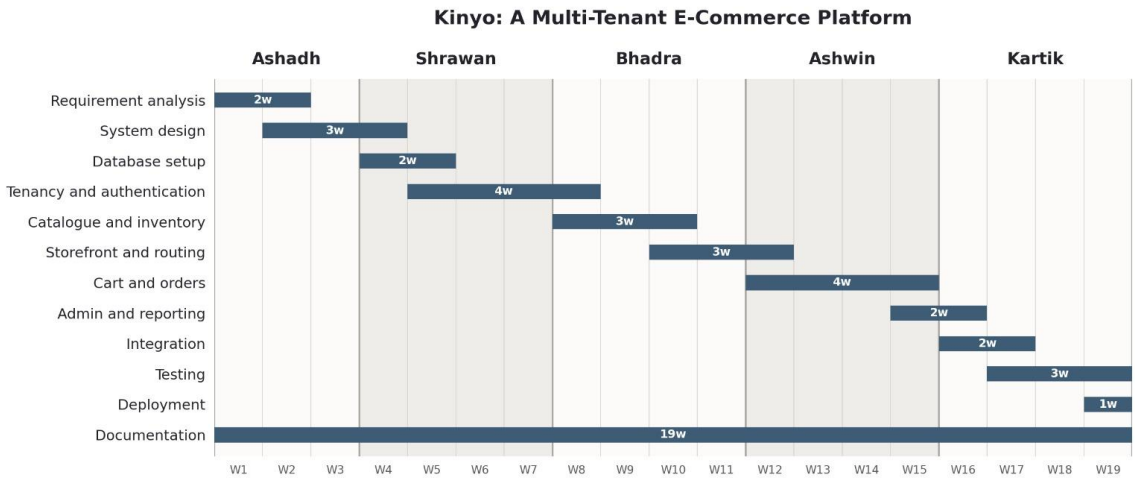
The project runs for nineteen weeks, from Ashadh to Kartik. Requirement analysis, design, development, integration, testing, deployment and documentation are planned as separate activities, and testing is allocated its own period rather than being absorbed into development. The schedule is shown in Figure 2.

Planned Activities

- Requirement analysis and system design, five weeks in total, so that the data model and the tenancy strategy are settled before any application code is written.
- Database design and migration setup, two weeks, followed by the tenancy and authentication core over four weeks. This is the highest-risk part of the design and is built first so that every later increment exercises it.

- Catalogue and inventory, three weeks, then the storefront and host-based routing, three weeks.
- Cart and orders, four weeks, then the seller dashboard, administration console and reporting, two weeks.
- Integration of the modules, two weeks, overlapping the end of development.
- Testing, three weeks, covering unit, integration and user acceptance testing, followed by one week for deployment and domain configuration.
- Documentation and report writing run across all nineteen weeks, so that the final report does not depend on a single concentrated effort at the end.

Figure 2
Gantt Chart



3.3 System Architecture

The system follows a three-tier architecture consisting of a presentation tier, an application tier and a data tier, shown in Figure 3. The tiers communicate only through defined interfaces, so that each can be developed and deployed separately.

The presentation tier is a Next.js application that serves three kinds of page from one deployment: the public storefronts, the seller dashboard and the platform administration console.

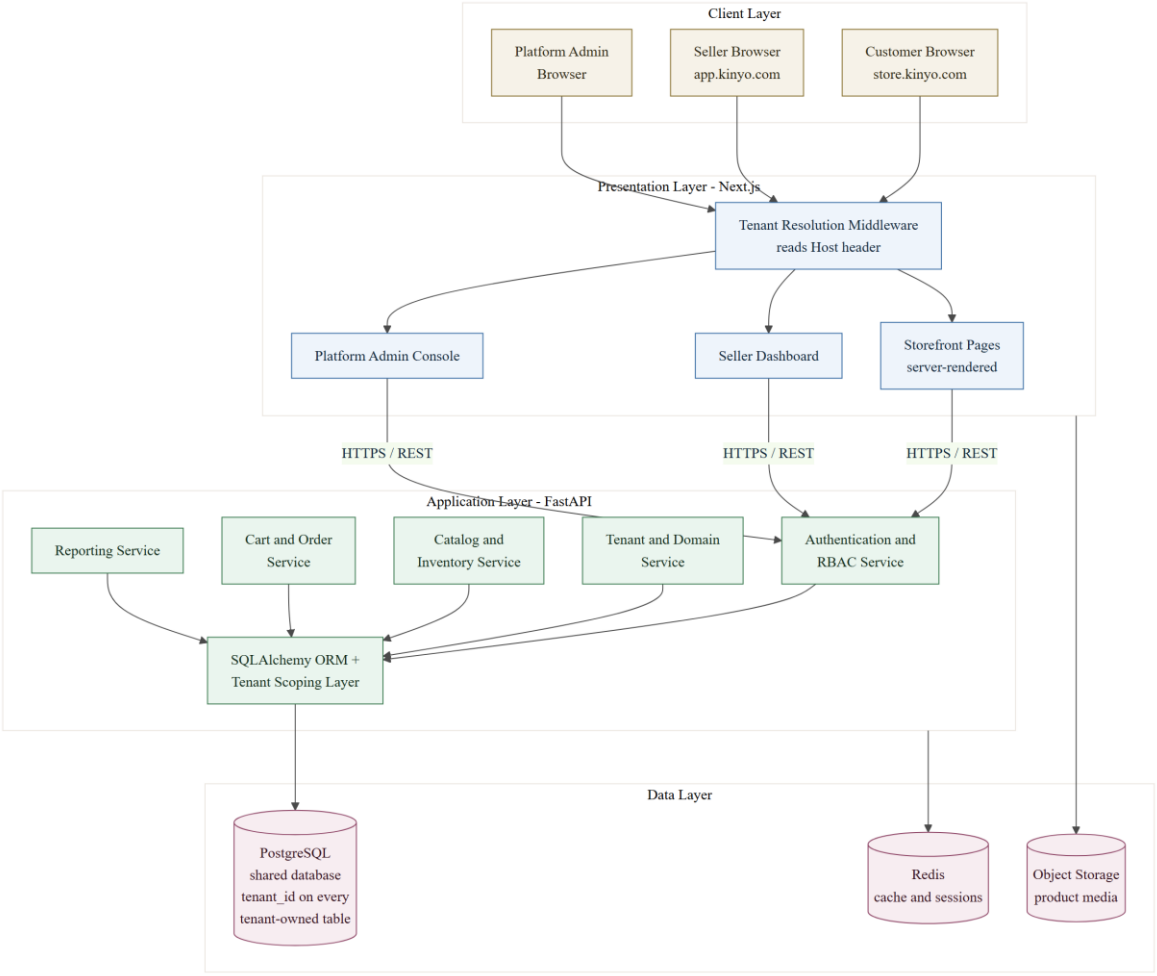
Every request first passes through tenant resolution middleware, which reads the HTTP Host header and looks up the corresponding store. A request for a platform subdomain such as a seller's store address, or for a verified custom domain, is resolved to that tenant and rewritten to the storefront routes; a request for the application subdomain is routed to the dashboard. Storefront pages are rendered on the server so that product pages are indexable by search engines, which is a functional requirement for sellers who depend on being found.

The application tier is a FastAPI service that exposes a versioned REST interface. It is organised into service modules for authentication and access control, tenant and domain management, catalogue and inventory, cart and orders, and reporting. Every module reaches the database through a single data-access layer built on the SQLAlchemy ORM. That layer holds the tenancy logic: the tenant established for the request is stored in a request-scoped context, and every query against a tenant-owned table is filtered by it, so that isolation does not depend on each individual query being written correctly.

The data tier is a single PostgreSQL database in which every tenant-owned table carries a tenant identifier column, supported by Redis for caching and session data and by object storage for product images. Row-level security policies in PostgreSQL provide a second line of defence behind the application-level scoping, so that a query that escapes the tenancy layer still cannot read another tenant's rows.

A typical request illustrates the flow. A customer opens a storefront address; the presentation tier resolves the host to a tenant and requests that tenant's published products from the application tier over HTTPS; the application tier authorises the request, applies the tenant filter in the data-access layer, and queries PostgreSQL; the resulting rows are serialised as JSON, returned to the presentation tier, and rendered into HTML that is sent to the browser.

Figure 3
System Architecture of the Kinyo Platform



3.4 Algorithm

Three parts of the system depend on logic that is not a simple create, read, update or delete operation. Each is described below as numbered steps.

3.4.1 Tenant Resolution Algorithm

This algorithm establishes which store a request belongs to. It runs before any other processing, in the presentation tier for page requests and in the application tier for interface requests.

1. Read the Host header of the incoming request and convert it to lower case, removing any port suffix.

2. If the host equals the platform application domain, mark the request as a dashboard request, resolve the tenant from the authenticated user's active membership, and stop.
3. If the host equals the platform root domain, mark the request as a marketing request with no tenant, and stop.
4. If the host ends with the platform storefront suffix, extract the leading label as the store slug and look up the store with that slug.
5. Otherwise, treat the host as a custom domain and look up a verified domain record with that host name.
6. If no store is found, or the store status is not active, return a store-not-found response and stop.
7. Store the identifier of the resolved tenant in the request-scoped context, load the theme settings for that tenant, and continue processing the request.

3.4.2 Cart Pricing and Discount Algorithm

This algorithm computes the amount payable for a cart. It is executed whenever the cart changes and again at checkout, so that a price displayed to the customer is recomputed rather than trusted from the client.

1. Set the subtotal to zero.
2. For each cart line, read the current price of the referenced variant, multiply it by the line quantity, store the result as the line total, and add it to the subtotal.
3. If a discount code has been supplied, retrieve the discount belonging to the current tenant with that code; if it does not exist, has expired, or its minimum order value exceeds the subtotal, reject the code and set the discount amount to zero.

4. If the discount is valid and of percentage type, set the discount amount to the subtotal multiplied by the percentage value, rounded to two decimal places; if it is of fixed type, set the discount amount to the lesser of the fixed value and the subtotal.
5. Determine the shipping zone that contains the delivery address of the order, and read the rate defined for that zone; if no zone matches, apply the default rate of the store.
6. Compute the total as the subtotal, minus the discount amount, plus the shipping rate.
7. Return the subtotal, the discount amount, the shipping rate and the total, together with the reason for any rejected discount code.

3.4.3 Order Placement and Stock Reservation Algorithm

This algorithm converts a cart into an order. It must not allow two customers to be sold the last unit of the same variant, so the stock check and the reservation are performed inside one database transaction.

1. Begin a database transaction.
2. Re-read every cart line and lock the inventory row of each referenced variant for update.
3. For each line whose variant is tracked, compare the requested quantity with the stock on hand minus the stock already reserved.
4. If any line fails the comparison, roll back the transaction and return the cart to the customer with a stock conflict message identifying the affected lines.
5. Recompute the cart totals using the algorithm in Section 3.4.2, so that the stored order reflects prices verified at the moment of placement.
6. Create the order record with a generated order number, the resolved tenant, the customer, the delivery address, the computed totals, a status of pending and a payment method of cash on delivery.

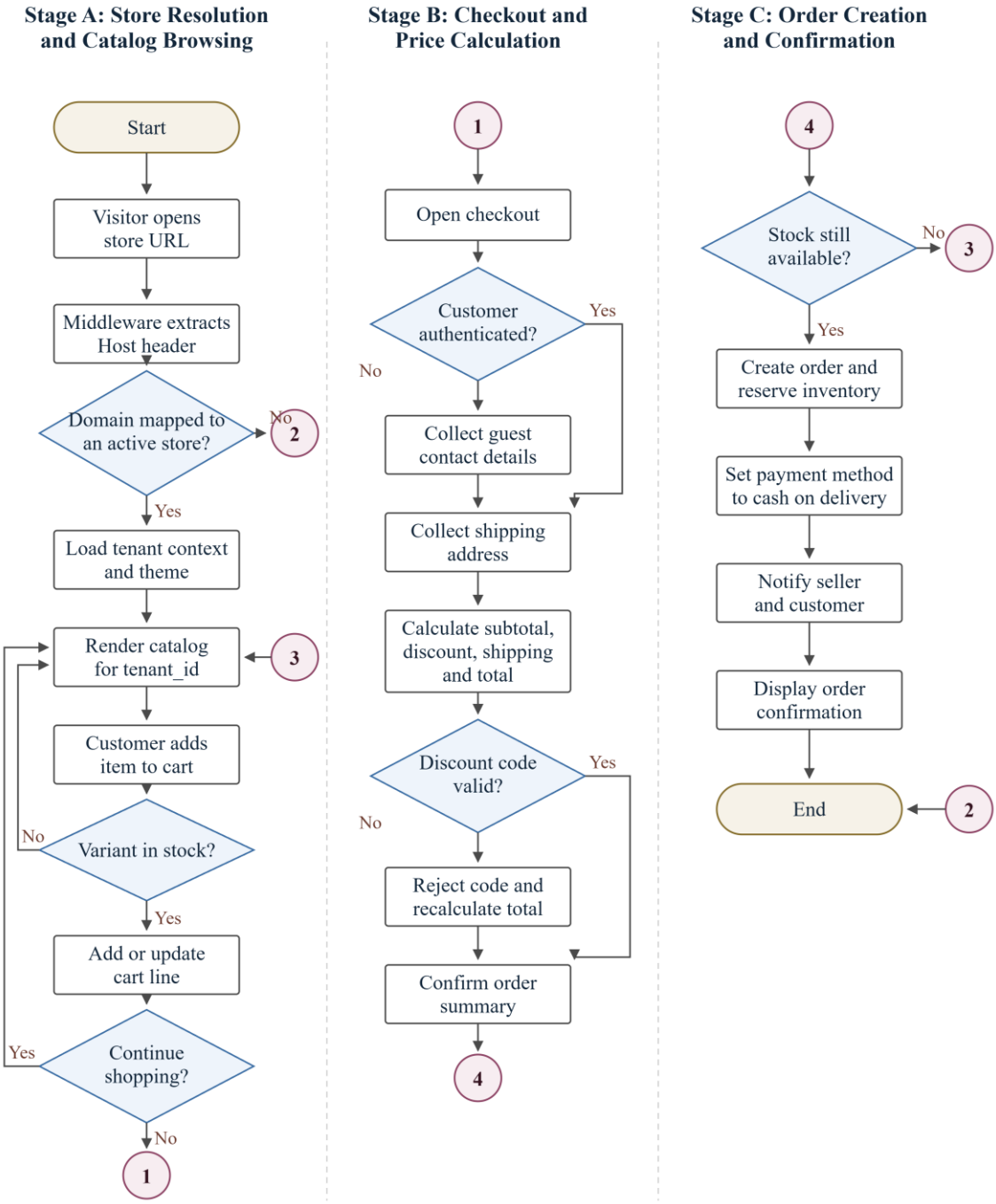
7. Create one order line for each cart line, copying the unit price into the line so that later price changes do not alter the order.
8. Increase the reserved quantity of each tracked variant by the ordered quantity.
9. Mark the cart as converted, commit the transaction, and queue notifications to the seller and the customer.

3.5 System Flowchart

Figure 4 shows the sequence of operations performed by the system from the moment a visitor opens a storefront address to the moment an order is confirmed. The chart is divided into three stages, connected by the numbered off-page connectors.

Figure 4

System Flowchart: Storefront Resolution to Order Confirmation



Stage A resolves the store and serves the catalogue. The middleware extracts the Host header and applies the tenant resolution algorithm of Section 3.4.1; if no active store matches, the visitor is shown a store-not-found page and the flow ends. Otherwise the tenant context and

theme are loaded and the catalogue for that tenant is rendered. When the customer adds an item, the system checks that the chosen variant is in stock before creating or updating the cart line, and the customer may continue browsing or proceed to checkout.

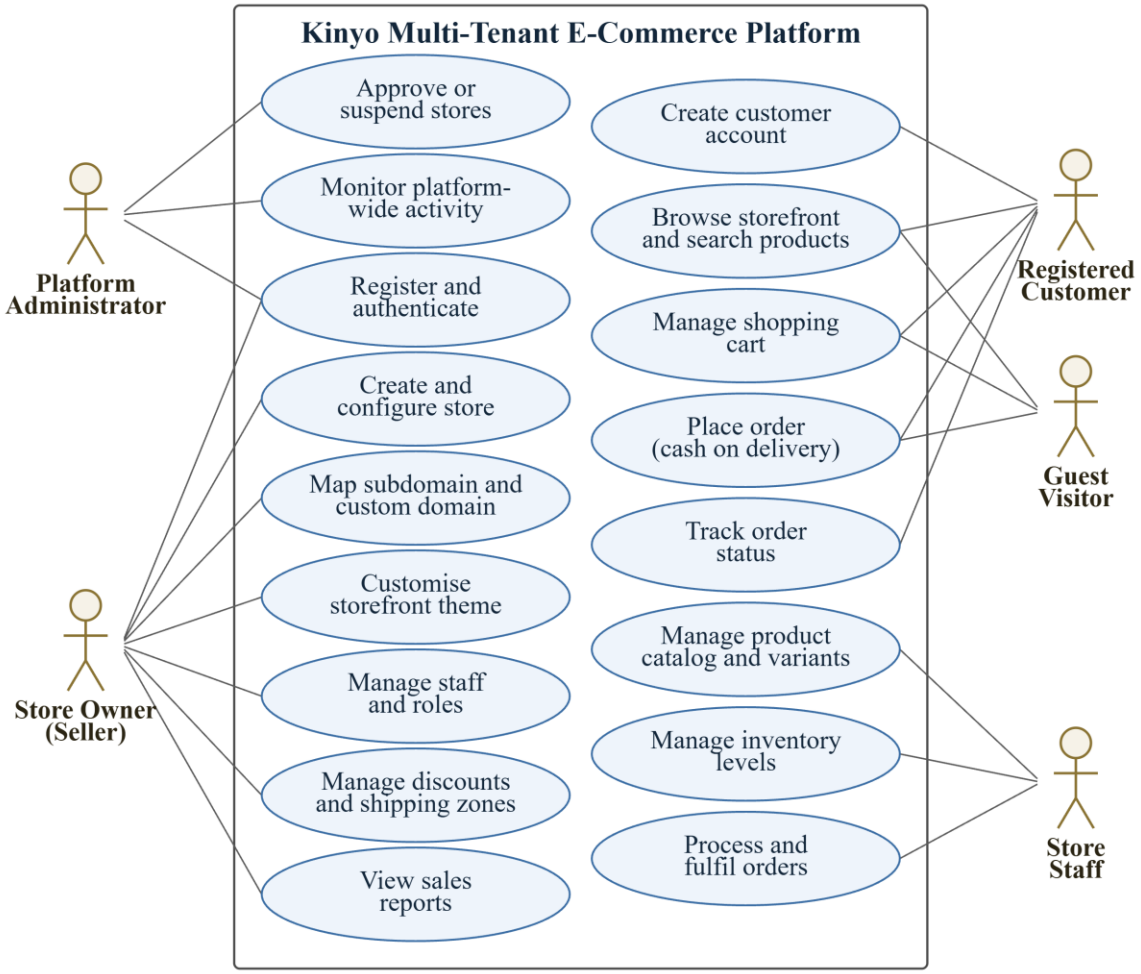
Stage B collects the information needed to complete the order and computes the amount payable. A customer who is not authenticated supplies contact details as a guest. The shipping address is collected, and the totals are computed by the algorithm of Section 3.4.2; an invalid discount code is rejected and the totals are recomputed before the order summary is confirmed.

Stage C creates the order. The stock check of Section 3.4.3 is repeated at this point, because stock may have changed while the customer was completing the checkout. If it fails, the customer is returned to the cart through connector 3. If it succeeds, the order is created and inventory is reserved in one transaction, the payment method is recorded as cash on delivery, the seller and customer are notified, and the confirmation is displayed.

3.6 Use Case Diagram

The use case diagram in Figure 5 shows the interactions between the actors and the system. Five actors are involved. The platform administrator approves and suspends stores and monitors platform-wide activity. The store owner creates and configures a store, maps its subdomain and custom domain, customises the storefront theme, defines discounts and shipping zones, manages staff and roles, and views sales reports. Store staff manage the product catalogue and its variants, maintain inventory levels, and process and fulfil orders. The registered customer creates a customer account, browses and searches the storefront, manages a shopping cart, places orders and tracks their status. The guest visitor may browse, manage a cart and place an order without creating an account.

Figure 5
Use Case Diagram



As shown in Figure 5, the system supports 17 primary use cases across five actor roles. Registration and authentication is shared between the platform administrator and the store owner, because both are platform users; storefront customers authenticate separately through the customer account use case, since customer accounts belong to a single tenant rather than to the platform.

3.7 Data Flow Diagram

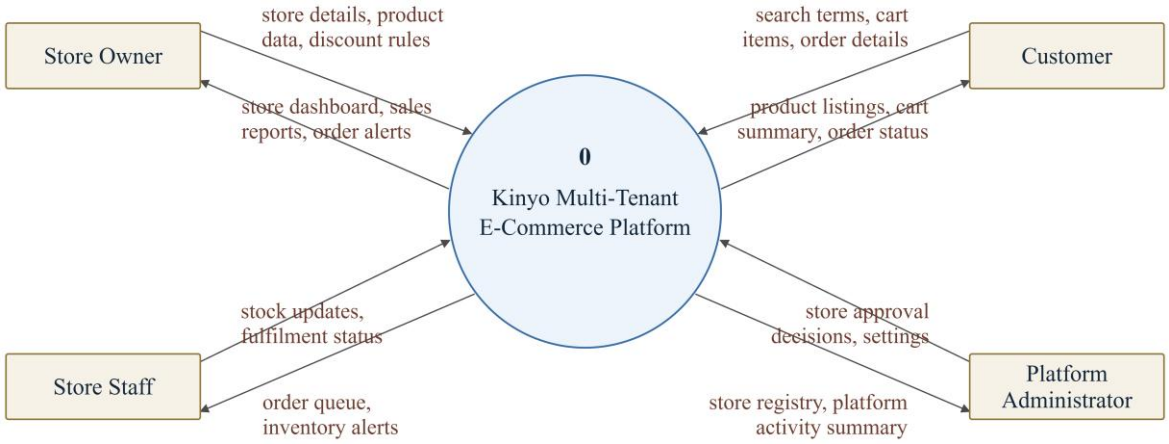
3.7.1 Level 0: Context Diagram

The context diagram in Figure 6 represents the whole platform as a single process and shows the data that crosses its boundary. Four external entities interact with the system. The

store owner supplies store details, product data, and discount and shipping rules, and receives the store dashboard, sales reports and order alerts. Store staff supply stock updates and fulfilment status, and receive the order queue and inventory alerts. The customer supplies search terms, cart items and order and address details, and receives product listings, cart summaries and order status. The platform administrator supplies store approval decisions and platform settings, and receives the store registry and a platform activity summary.

Figure 6

Data Flow Diagram: Level 0 (Context Diagram)

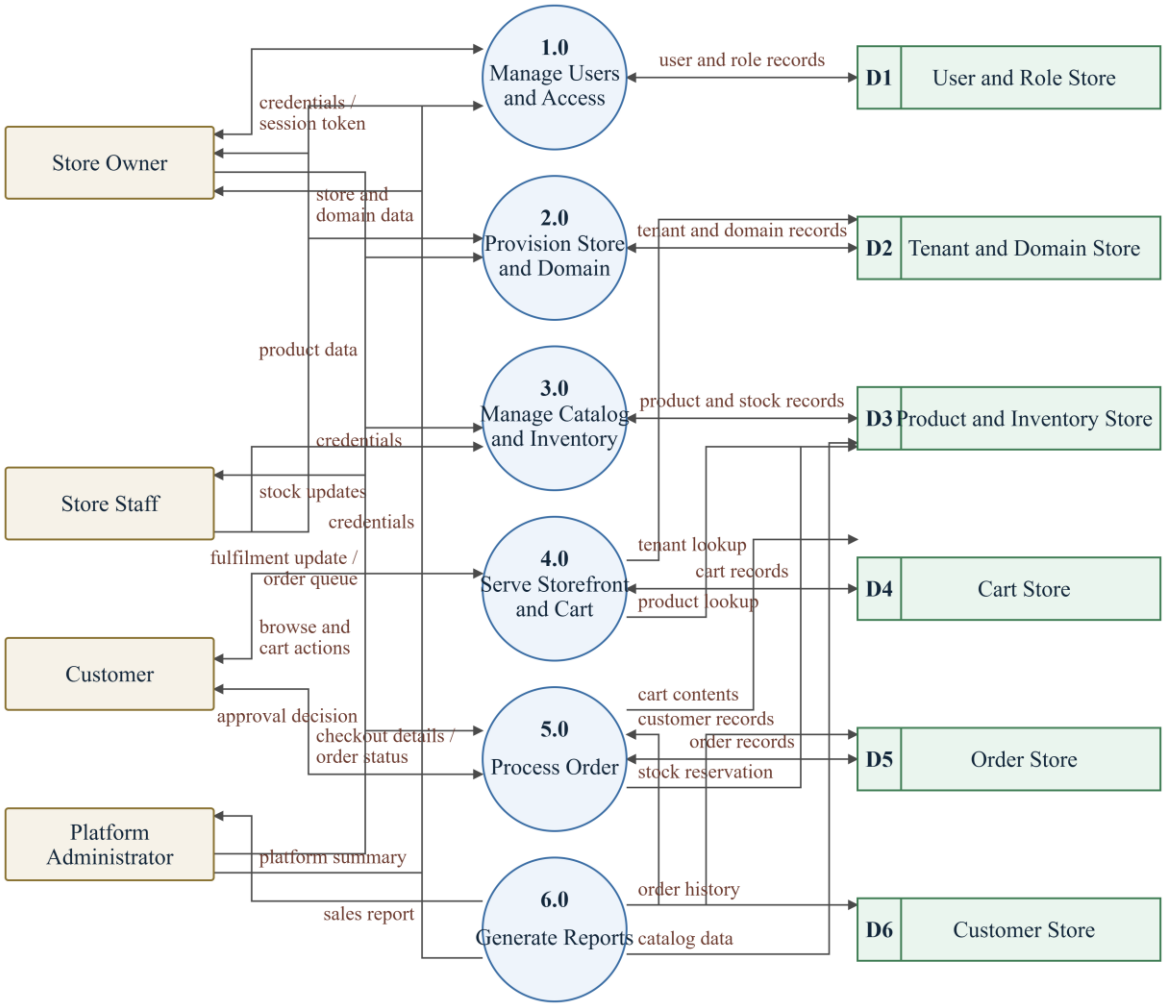


3.7.2 Level 1: Detailed Data Flow Diagram

Figure 7 decomposes the single process of Figure 6 into six processes and six data stores. Process 1.0, Manage Users and Access, authenticates platform users and issues session tokens against the user and role store. Process 2.0, Provision Store and Domain, creates tenant records and domain mappings and applies the administrator's approval decisions. Process 3.0, Manage Catalog and Inventory, maintains products, variants and stock levels. Process 4.0, Serve Storefront and Cart, resolves the tenant, reads the catalogue and maintains cart records for storefront visitors. Process 5.0, Process Order, converts a cart into an order, reserves stock, records the customer, and reports fulfilment progress. Process 6.0, Generate Reports, reads order

and catalogue data to produce sales reports for sellers and an activity summary for administrators.

Figure 7
Data Flow Diagram: Level 1



The data stores mirror the entity groups of Section 3.8: D1 holds users and roles, D2 tenants and domains, D3 products and inventory, D4 carts, D5 orders and D6 storefront customers. Every store except D1 is partitioned internally by tenant identifier.

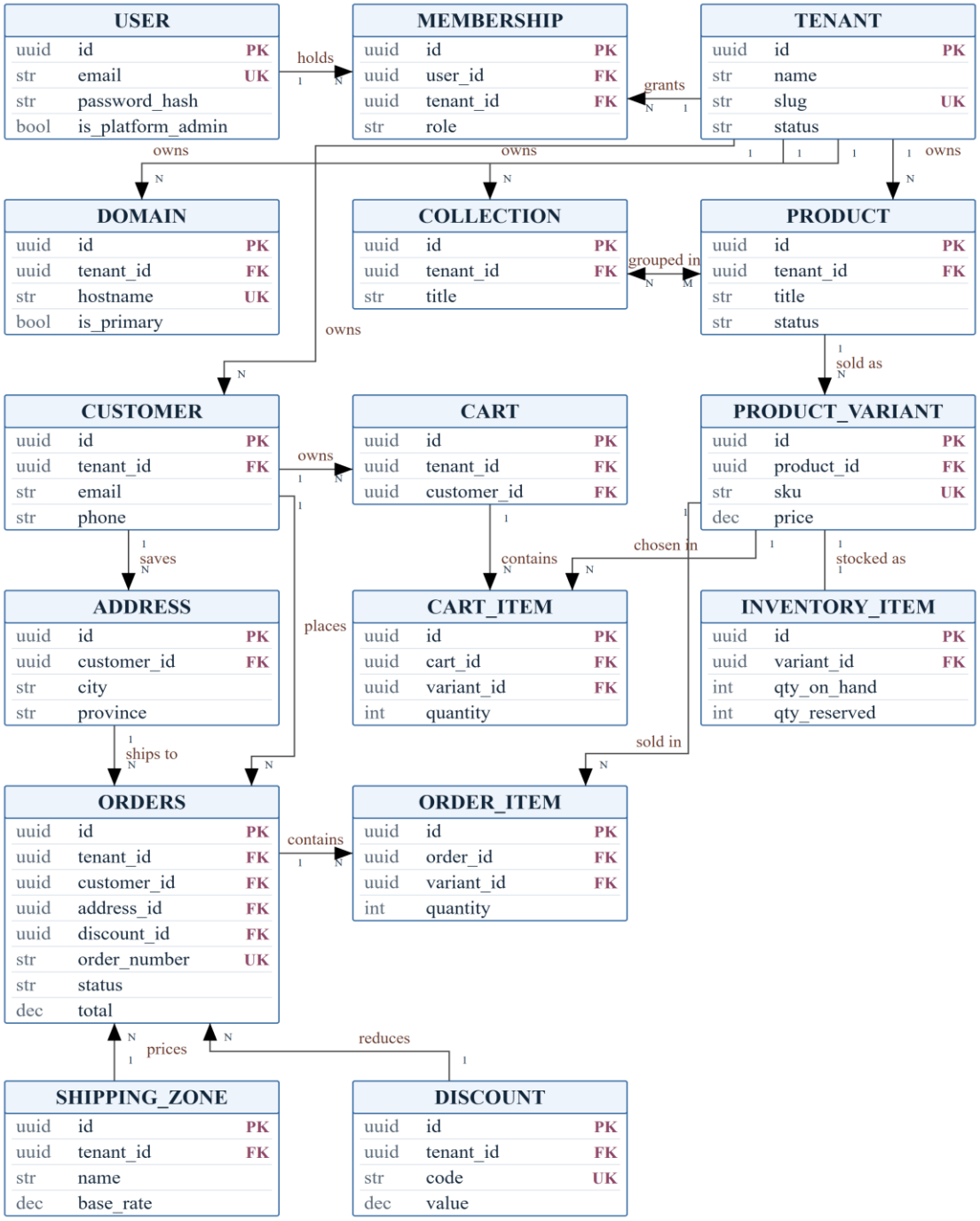
3.8 Entity Relationship Diagram

The entity relationship diagram in Figure 8 shows the principal entities of the database, their primary and foreign keys and the cardinality of each relationship. TENANT is the root of

the tenant-owned data: a tenant has many domains, products, collections, customers, carts, orders, discounts and shipping zones, and every one of those tables carries tenant_id as a foreign key referencing TENANT. Platform users are held separately in USER and are connected to tenants through MEMBERSHIP, which carries the role held by that user in that tenant; this allows one person to hold different roles in different stores.

Figure 8

Entity Relationship (ER) Diagram



In the catalogue, a PRODUCT has many PRODUCT_VARIANT rows, each identified by a unique SKU and carrying its own price, and each variant has exactly one INVENTORY_ITEM recording stock on hand and stock reserved. Products are grouped into

collections through a many-to-many association. In the commerce path, a CUSTOMER saves many addresses and owns carts and orders; a CART contains many CART_ITEM rows and an ORDERS row contains many ORDER_ITEM rows, each referring to a variant and storing the unit price agreed at the time. An order also references the address it ships to, the discount applied and the shipping zone that priced it.

Supporting tables that are not drawn in Figure 8, in order to keep the diagram legible, are the theme settings of a store, product media assets, shipment tracking records and the platform-level role definitions. Each follows the same rule as the entities shown: it carries tenant_id where it is tenant-owned, and its primary key is a universally unique identifier.

Chapter 4: Implementation Plan

This chapter describes how the project will be implemented. Since the project is still at the proposal stage, no implementation or testing results are reported here.

4.1 Hardware and Software Requirements

This section states the hardware and software required to develop, test and deploy the system.

Hardware Requirements

1) Processor

- Minimum: Intel Core i5 (8th generation) or an equivalent AMD Ryzen 5. Sufficient for running the application server, the database and the front-end build together.
- Recommended: Intel Core i7 or Ryzen 7, which shortens front-end build times and makes running the full stack in containers comfortable.

2) Memory

- Minimum: 8 GB. Enough to run PostgreSQL, Redis, the FastAPI service and the Next.js development server at the same time.
- Recommended: 16 GB, which leaves room for the browser, the editor and the container runtime without swapping.

3) Storage

- A solid state drive is recommended, because dependency installation and front-end builds are dominated by small-file input and output.
- Minimum 40 GB free for source code, dependencies, container images and the local database.

4) Display and Network

- A display of 1920 by 1080 or higher, which makes dashboard and storefront layout work practical.
- A broadband internet connection for package installation, deployment and domain verification.

5) Deployment Server

- One application instance of 1 virtual CPU and 1 GB of memory, which is within the free tier of the hosting platforms considered.
- A small managed PostgreSQL instance for the shared database.
- No graphics processor is required at any stage, because the system performs no model training or heavy computation.

Software Requirements

1) Operating System

- Any modern 64-bit operating system. Windows 10 or 11, or Ubuntu 22.04 LTS, are used by the team and are both supported by every tool in the stack.

2) Back End

- Python 3.11 as the implementation language.
- FastAPI with Pydantic and Uvicorn for the REST interface, request validation and generated interface documentation.
- SQLAlchemy 2.0 for object-relational mapping, which is where the tenant scoping is applied, and Alembic for versioned schema migrations.

3) Front End

- TypeScript as the implementation language.

- Next.js 15 with React for server-rendered storefronts, the seller dashboard and the platform administration console.
- Tailwind CSS for styling, which keeps the storefront themes to a small set of design tokens.

4) Data and Storage

- PostgreSQL 16 as the relational database, chosen for its row-level security policies and JSONB columns.
- Redis 7 for cached storefront responses and session data.
- An S3-compatible object store for product images and theme assets.

5) Development and Testing Tools

- Visual Studio Code as the development environment.
- Docker and Docker Compose for a reproducible local database and cache.
- pytest for unit and integration testing, and Playwright for end-to-end tests through a real browser.
- Git and GitHub for source control and collaboration.

4.2 Proposed Technology Stack

The proposed technology stack is set out below, tier by tier, with the reason for each major choice stated alongside it.

1) Presentation Tier

- Next.js 15 with React and Tailwind CSS, served over HTTPS.
- Chosen because its middleware runs before routing and can rewrite a request based on the Host header, which is exactly the mechanism required to serve many storefronts and two

applications from one deployment, and because server-side rendering produces complete HTML for storefront pages so that they are indexable by search engines.

2) Application Tier

- FastAPI with Pydantic and Uvicorn, exposing a versioned REST interface.
- Chosen because it validates every request and response against declared models and generates interface documentation from those same models, so the contract cannot drift from the code. Its asynchronous request handling also suits a workload dominated by database waits. Django was considered but its session-based, single-tenant conventions would have to be worked against rather than with.

3) Data Tier

- PostgreSQL 16 as the single shared database, with Redis 7 for caching and an S3-compatible object store for media.
- PostgreSQL was chosen over MySQL because the tenancy design depends on features it provides directly: row-level security policies give a database-level guarantee of isolation behind the application filter, and JSONB columns hold variant options and theme settings without a separate table for every attribute.

4) Data Access

- SQLAlchemy 2.0 with Alembic migrations.
- Chosen because the tenant filter can be implemented once, in the session and query layer, and applied to every tenant-owned model rather than repeated in each query. Alembic keeps the schema under version control alongside the code.

5) Authentication

- JSON Web Tokens for session handling and Argon2 for password hashing.

- Argon2 was chosen over older algorithms because it is the current recommendation for password storage in the OWASP guidance.

6) Deployment

- Vercel for the presentation tier and Render for the application tier and the managed database.
- Both offer free tiers sufficient for a demonstration deployment, and both support the custom domain mapping the storefronts require.

4.3 Proposed Testing and Development Approach

The repository layout, testing approach and version control practice are described below.

- **Project structure:** the system is held in two repositories. The back-end repository contains app/ (api/, core/, models/, schemas/, services/), alembic/ for migrations, tests/ and docs/. The front-end repository contains src/app/ with separate route groups for the storefront, the seller dashboard and the platform console, src/components/, src/lib/ and the tenant resolution middleware.
- **Unit testing:** each service module is tested in isolation with pytest, using a transactional test database. Pricing, discount and stock reservation logic are covered by table-driven cases including boundary values such as a zero-stock variant and an expired discount code.
- **Integration testing:** the interface is exercised end to end against a live test database, covering registration, store provisioning, catalogue creation, cart operations and order placement. A dedicated group of integration tests creates two tenants and asserts that every read and write operation performed under one tenant's credentials fails or returns empty for the other tenant's data.

- **User acceptance testing:** a small group of prospective sellers and customers will be asked to complete defined tasks, such as creating a store, publishing a product and placing an order, and the outcome of each task will be recorded.
- **Test case documentation:** test cases are recorded in a table with the columns test case identifier, module, precondition, input, expected output, actual result and status. The 40 test cases named in the third specific objective are distributed across authentication and access control, tenant isolation, catalogue and inventory, cart and pricing, order placement, and storefront routing.
- **Version control:** all source code and design documents are maintained in Git from the start of the project, with a branch for each feature and a review before merging. Migrations are committed together with the code that requires them.

Git repositories:

- **Back end:** https://github.com/HikmatBaniya/kinyo
- **Front end:** https://github.com/HikmatBaniya/kinyo-front

The repository links above are maintained from the start of the project, as required by the proposal checklist.

Chapter 5: Expected Outcomes

This chapter describes the anticipated results and benefits of the proposed project. Everything stated here is an expectation to be verified during testing, not a result already obtained.

5.1 Expected System Deliverables

The project is expected to deliver the following.

- A deployed multi-tenant web application in which a seller can register, create a store, publish a catalogue and receive orders, and in which a customer can browse a storefront and place an order using cash on delivery.
- A seller dashboard for catalogue, inventory, discount, shipping, staff and order management, and a platform administration console for approving and suspending stores.
- Server-rendered storefronts reachable at platform subdomains and at verified custom domains, each rendering only the catalogue and theme of its own tenant.
- The complete source code of the back-end and front-end applications, together with setup instructions, interface documentation generated from the code, and the design documents produced in Chapter 3.
- The database schema as a versioned sequence of migrations, together with a seed dataset of sample stores, products and orders for demonstration.
- A documented test suite and a completed test case table recording the outcome of each of the defined test cases, including the cross-tenant isolation cases.
- The Git repositories containing all code, migrations and documentation, with the history of the project preserved.

5.2 Expected System Features and Benefits

The completed system is expected to allow an independent seller to move from a social media page to a branded storefront without technical assistance. Because store provisioning is self-service, the steps that currently require a developer, namely obtaining hosting, installing a commerce package and configuring a domain, are expected to be replaced by a registration form and a domain verification step.

The system is expected to remove the manual order book described in Section 1.2. Orders will be recorded with a generated order number, a stored delivery address, the prices agreed at the time of purchase and an explicit status, so that the duplication and loss of orders that arises from tracking sales in chat threads is eliminated. This expectation will be verified during user acceptance testing by asking sellers to complete a defined set of order-handling tasks and recording the outcome.

Because stock is reserved inside the same transaction that creates the order, the system is expected to prevent the sale of items that are no longer available, which the current manual process cannot detect until the seller attempts to dispatch the goods.

For customers, the expected benefit is a storefront that shows accurate availability, a consistent price including delivery charges, and an order status that can be checked without contacting the seller.

For the platform operator, the expected benefit of the shared-database multi-tenant design is that the marginal cost of an additional seller is the cost of the rows that seller creates, rather than the cost of a further deployment, which is what makes an affordable service for single-shop retailers possible.

The seller is also expected to gain visibility that a social media page cannot provide. Because storefront pages are rendered on the server and served from a domain the seller controls, product pages are expected to be indexable by search engines, so that a customer searching for a product can reach the seller's store directly rather than only through a social media feed.

Finally, the project is expected to produce a documented reference implementation of shared-schema multi-tenancy in a Python and TypeScript stack, including the tenancy layer and the isolation test cases, which may be reused by later projects addressing similar problems.

References

Adobe. (2024). *Adobe Commerce developer documentation* [Computer software documentation]. https://developer.adobe.com/commerce/docs/

Bezemer, C.-P., & Zaidman, A. (2010). Multi-tenant SaaS applications: Maintenance dream or nightmare? In *Proceedings of the Joint ERCIM Workshop on Software Evolution and International Workshop on Principles of Software Evolution* (pp. 88–92). Association for Computing Machinery. https://doi.org/10.1145/1862372.1862393

Chong, F., & Carraro, G. (2006). *Architecture strategies for catching the long tail*. Microsoft Corporation. https://learn.microsoft.com/en-us/previous-versions/dotnet/articles/aa479069(v=msdn.10)

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387. https://doi.org/10.1145/362384.362685

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

International Trade Administration. (2024). *Nepal country commercial guide: eCommerce*. U.S. Department of Commerce. https://www.trade.gov/country-commercial-guides/nepal-ecommerce

Krebs, R., Momm, C., & Kounev, S. (2012). Architectural concerns in multi-tenant SaaS applications. In *Proceedings of the 2nd International Conference on Cloud Computing and Services Science* (pp. 426–431). SciTePress. https://doi.org/10.5220/0003957604260431

Medusa. (2024). *Medusa documentation* [Computer software documentation]. https://docs.medusajs.com/

Open Web Application Security Project. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation* [Computer software documentation]. https://www.postgresql.org/docs/16/

Ramírez, S. (2024). *FastAPI documentation* (Version 0.115) [Computer software documentation]. https://fastapi.tiangolo.com/

Saleor Commerce. (2024). *Saleor documentation* [Computer software documentation]. https://docs.saleor.io/

Shopify. (2024). *Shopify developer documentation* [Computer software documentation]. https://shopify.dev/docs

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

SQLAlchemy Project. (2024). *SQLAlchemy 2.0 documentation* [Computer software documentation]. https://docs.sqlalchemy.org/en/20/

Vercel. (2024). *Next.js documentation* (Version 15) [Computer software documentation]. https://nextjs.org/docs

Wix.com. (2024). *Wix eCommerce developer documentation* [Computer software documentation]. https://dev.wix.com/docs

WooCommerce. (2024). *WooCommerce documentation* [Computer software documentation]. https://woocommerce.com/documentation/